

# Enterprise React Frontend

## Ana Katmanlar

### Profesyonel CRM / ERP sistemleri için uzun ömürlü frontend platform mimarisi

**Amaç:** React uygulamasına yalnızca yeni feature eklemek değil; yeni feature ekleme maliyetini sürekli düşüren, güvenli, ölçeklenebilir ve genişletilebilir bir Enterprise Frontend Platform kurmak.

|                |             |               |
|----------------|-------------|---------------|
| 60             | 8           | 5             |
| ana capability | mimari grup | uygulama fazı |

**Temel prensip:** Feature bazlı hackler yerine Command, Action, Permission, Query, Mutation, Form, Grid, Extension ve Telemetry gibi tekrar kullanılabilir platform capabilityleri oluşturulur.

### Mimari Gruplar

- 1. Platform, Modülerlik ve Genişletilebilirlik** - Uygulamanın core iskeleti ve yeni modüllerin core koda minimum dokunuşla eklenmesini sağlayan temel katmanlar.
- 2. API, Server State, Mutation ve Realtime** - Frontend ile .NET/API arasındaki veri yaşam döngüsünü güvenli, hızlı ve izlenebilir hale getiren katmanlar.
- 3. Yetki, Tenant, Feature ve Configuration** - Multi-tenant ve enterprise ürün varyasyonlarını merkezi policy/capability yaklaşımıyla yöneten altyapı.
- 4. Form, Grid ve Veri Çalışma Alanı** - ERP kullanıcılarının gün boyu kullandığı form, tablo, filtre, selection ve workspace deneyiminin platformlaştırılması.
- 5. Bildirim, Arka Plan İşleri ve Design System** - Uzun işlemler, tutarlı UI davranışı, erişilebilirlik ve uluslararası kullanım için temel ürün altyapısı.
- 6. Güvenlik, Audit, Observability ve Performans** - Enterprise güven, production görünürlüğü ve uzun vadeli performans için zorunlu kalite katmanları.
- 7. Test, Developer Experience ve Delivery Governance** - Büyük ekip ve sık release temposunda mimariyi, kaliteyi ve dağıtım güvenliğini koruyan sistemler.
- 8. White-Label, Offline, Veri Taşıma ve AI Hazırlığı** - Ürünü partner/tenant varyasyonlarına, zayıf bağlantıya, yüksek hacimli veri taşımaya ve gelecekte agent tabanlı kullanıma hazırlayan katmanlar.

## 1. Platform, Modülerlik ve Genişletilebilirlik

Uygulamanın core iskeleti ve yeni modüllerin core koda minimum dokunuşla eklenmesini sağlayan temel katmanlar.

| # | Katman                             | Çalışma Mantığı                                                                                                            | Enterprise Katkısı                                                                         |
|---|------------------------------------|----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| 1 | <b>App Shell</b>                   | Header, sidebar, workspace, in-app tabs, drawers, modals ve global bildirimlerin tek uygulama kabuğunda orkestre edilmesi. | Tüm modüllerin aynı davranış ve yaşam döngüsünü paylaşmasını sağlar.                       |
| 2 | <b>Module Architecture</b>         | CRM, satış, ürün, stok, finans gibi domainlerin bağımsız modüller ve açık sınırlar halinde organize edilmesi.              | Repo büyüdükçe coupling ve spaghetti bağımlılıkları kontrol altında kalır.                 |
| 3 | <b>Route Registry</b>              | Route tanımlarının merkezi/declarative registry üzerinden eklenmesi.                                                       | Yeni modül ve extensionlar core router kodunu sürekli değiştirmeden sisteme bağlanabilir.  |
| 4 | <b>Navigation Registry</b>         | Menü ve navigasyon öğelerinin hard-code edilmek yerine registry/config üzerinden üretilmesi.                               | Tenant, paket, permission ve extension bazlı navigasyon mümkün olur.                       |
| 5 | <b>Command Registry</b>            | Cmd+K dahil tüm komutların tek registry ile tanımlanması ve farklı UI yüzeylerinden çağrılması.                            | Aynı komut palette, shortcut, toolbar ve ileride AI tarafından tekrar kullanılabilir.      |
| 6 | <b>Action Registry</b>             | Edit, approve, delete, export, assign gibi business aksiyonlarının ortak bir action contract ile tanımlanması.             | Aynı iş kuralı row action, bulk action, toolbar ve command palette içinde tekrar yazılmaz. |
| 7 | <b>Extension / Slot System</b>     | customer.details.tabs veya invoice.sidebar gibi açık extension pointleri ve slot kontratları.                              | Müşteri/partner geliştirmeleri core ERP koduna minimum dokunuşla takılabilir.              |
| 8 | <b>Feature Capability Registry</b> | Bir capability veya featurein runtime sırasında mevcut olup olmadığının merkezi çözülmesi.                                 | Tenant, paket ve müşteri farklılıkları if/else dağınıklığına dönüşmez.                     |

## 2. API, Server State, Mutation ve Realtime

Frontend ile .NET/API arasındaki veri yaşam döngüsünü güvenli, hızlı ve izlenebilir hale getiren katmanlar.

| # | Katman                 | Çalışma Mantığı                                                          | Enterprise Katkısı                                                      |
|---|------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------------|
| 9 | <b>Typed API Layer</b> | Backend API erişiminin generated/typed client veya ortak bir API katmanı | Endpoint contractları componentlere dağılmaz; refactor ve hata yakalama |

| #  | Katman                           | Çalışma Mantığı                                                                                                    | Enterprise Katkısı                                                               |
|----|----------------------------------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
|    |                                  | üzerinden yapılması.                                                                                               | kolaylaştır.                                                                     |
| 10 | Query Architecture               | Query key factory, caching, stale policy, invalidation ve prefetch kurallarının standartlaştırılması.              | Server state tutarlılığı ve performans kontrol edilebilir hale gelir.            |
| 11 | Server / Client State Separation | API verisi, local UI state, form state ve global app statein net ayrılması.                                        | Context/Redux/Zustand/query cache karmaşasının önüne geçilir.                    |
| 12 | Mutation Infrastructure          | Validate -> execute -> reconcile/invalidate -> telemetry -> undo/rollback yaşam döngüsünün standartlaştırılması.   | Tüm CRUD ve business aksiyonları aynı güvenli mutation modeline oturur.          |
| 13 | Optimistic Update Layer          | Uygun ve geri alınabilir işlemlerde server cevabı beklenmeden UI güncellenir; hata halinde rollback/retry yapılır. | Algılanan hız ciddi artar; pending/failed/committed durumları ile güven korunur. |
| 14 | Error Architecture               | Global, route, component ve async Error Boundary katmanlarının merkezi hata modeliyle tasarlanması.                | Tek bir servis veya component hatası tüm uygulamayı çökertmez.                   |
| 15 | Resilient Request Layer          | Timeout, retry, cancellation, deduplication, stale request ve network degradation yönetimi.                        | Zayıf ağ ve geçici backend problemlerinde ürün kontrollü davranır.               |
| 16 | Real-Time Event Layer            | WebSocket/SignalR/SSE üzerinden event bus, cache invalidation ve gerektiğinde canlı veri senkronizasyonu.          | Aynı kayıt üzerinde çalışan ekiplerin ekranları güncel kalır.                    |

### 3. Yetki, Tenant, Feature ve Configuration

Multi-tenant ve enterprise ürün varyasyonlarını merkezi policy/capability yaklaşımıyla yöneten altyapı.

| #  | Katman              | Çalışma Mantığı                                                                                         | Enterprise Katkısı                                                  |
|----|---------------------|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| 17 | Permission Engine   | Route, action, component, field ve veri görünürlüğü seviyesinde merkezi permission/capability kontrolü. | RBAC/ABAC gereksinimleri UI içine dağınık koşullar olarak gömülmez. |
| 18 | Tenant Context      | Tenant, organization, branch, environment ve aktif çalışma alanı bağlamının standartlaştırılması.       | Multi-tenant davranışlar component bazlı özel kodlara dönüşmez.     |
| 19 | Feature Flag Engine | Environment, tenant, kullanıcı                                                                          | Büyük değişiklikler kontrollü                                       |

| #  | Katman                      | Çalışma Mantığı                                                                                   | Enterprise Katkısı                                                                     |
|----|-----------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
|    |                             | veya cohort bazında feature flag ve rollout yönetimi.                                             | açılır ve gerektiğinde hızlıca geri alınır.                                            |
| 20 | <b>Configuration Engine</b> | UI davranışlarının güvenli bir kısmının deploy gerektirmeden configuration üzerinden yönetilmesi. | Enterprise müşteri uyarlamaları ve paket farklılıkları daha düşük maliyetle yönetilir. |

## 4. Form, Grid ve Veri Çalışma Alanı

ERP kullanıcılarının gün boyu kullandığı form, tablo, filtre, selection ve workspace deneyiminin platformlaştırılması.

| #  | Katman                          | Çalışma Mantığı                                                                                                | Enterprise Katkısı                                                                            |
|----|---------------------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 21 | <b>Schema-Driven Forms</b>      | Form alanları, layout, validation ve visibility kurallarının ortak schema/model üzerinden üretilebilmesi.      | Büyük ERP formlarının tekrarlı kodu ve müşteriye özel varyasyon maliyeti azalır.              |
| 22 | <b>Validation Architecture</b>  | Client ve server validation sonuçlarının ortak hata contractı ve alan eşleme standardı.                        | Her form farklı hata UX'i üretmez; backend iş kuralları tutarlı gösterilir.                   |
| 23 | <b>Draft Engine</b>             | Dirty state, autosave, local draft, restore, failed-save ve navigation guard davranışlarının merkezi yönetimi. | Uzun formlarda veri kaybı ve kaydetme stresi azalır.                                          |
| 24 | <b>Field Registry</b>           | Money, date, quantity, customer, product, ERP code gibi domain alanlarının standart field componentleri.       | Aynı veri tipi tüm sistemde aynı format, validation ve interaction davranışını taşır.         |
| 25 | <b>Enterprise Grid Layer</b>    | Kullanılan grid kütüphanesinin üzerine kendi DataGrid abstraction ve domain adapter katmanının kurulması.      | Vendor lock-in azalır; tüm modüllerde tutarlı grid davranışı sağlanır.                        |
| 26 | <b>Filter AST / Query Model</b> | Filtrelerin serializable bir AST/JSON modeliyle temsil edilmesi.                                               | Saved View, API filtering, URL state ve gelecekte semantic/AI filtering aynı modele bağlanır. |
| 27 | <b>View State Engine</b>        | Columns, sort, filters, density, grouping, page size ve görünüm durumunun ortak modelde tutulması.             | Tablo davranışları kullanıcı/ekip bazında kaydedilebilir ve taşınabilir olur.                 |
| 28 | <b>Selection Engine</b>         | Page selection, select-all-results, excluded rows ve büyük dataset bulk selection davranışları.                | On binlerce kayıtlı çalışan enterprise toplu işlemler güvenli hale gelir.                     |

| #  | Katman            | Çalışma Mantığı                                                                                        | Enterprise Katkısı                                               |
|----|-------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 29 | Overlay Manager   | Drawer, modal, popover, dialog ve floating panel katmanlarının tek orkestrasyon modeliyle yönetilmesi. | Nested modal, focus trap ve z-index kaosu azaltılır.             |
| 30 | Workspace Manager | Açık kayıtlar, in-app tabs, draftlar, aktif entity ve workspace restore durumlarının yönetimi.         | ERP klasik SPA yerine masaüstü çalışma ortamı gibi davranabilir. |

## 5. Bildirim, Arka Plan İşleri ve Design System

Uzun işlemler, tutarlı UI davranışı, erişilebilirlik ve uluslararası kullanım için temel ürün altyapısı.

| #  | Katman                   | Çalışma Mantığı                                                                                                     | Enterprise Katkısı                                                           |
|----|--------------------------|---------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| 31 | Notification Center      | Geçici toastlardan ayrı, kalıcı ve okunabilir notification/activity merkezi.                                        | Kullanıcı uzun süren veya kaçırdığı sistem olaylarını tekrar görebilir.      |
| 32 | Background Jobs UX       | Import, export, rapor, sync ve toplu işlemler için frontend job state, progress ve sonuç modeli.                    | Kullanıcı ekranı bloke etmeden uzun işlemleri takip edebilir.                |
| 33 | Design Token System      | Spacing, typography, semantic color, radius, elevation ve density tokenlarının standardizasyonu.                    | UI bütünlüğü ve tema/white-label kontrolü merkezi hale gelir.                |
| 34 | Enterprise Design System | Button seviyesinin ötesinde EntityPicker, MoneyField, StatusBadge, DataGrid, FormSection gibi domain componentleri. | Yeni feature geliştirme hızı artar ve iş ekranları daha tutarlı olur.        |
| 35 | Accessibility Foundation | Keyboard navigation, focus management, ARIA, screen reader ve contrast standartlarının temele alınması.             | Sonradan yamamak yerine kalıcı erişilebilirlik ve power-user desteği sağlar. |
| 36 | Localization Engine      | Dil, timezone, currency, number/date formatları ve locale-aware parsing altyapısı.                                  | Yeni ülke ve enterprise organizasyonlara açılım kolaylaşır.                  |

## 6. Güvenlik, Audit, Observability ve Performans

Enterprise güven, production görünürlüğü ve uzun vadeli performans için zorunlu kalite katmanları.

| #  | Katman         | Çalışma Mantığı           | Enterprise Katkısı |
|----|----------------|---------------------------|--------------------|
| 37 | Security Layer | CSP uyumu, XSS savunması, | Kurumsal güvenlik  |

| #  | Katman                    | Çalışma Mantığı                                                                                            | Enterprise Katkısı                                                                |
|----|---------------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
|    |                           | güvenli HTML/render politikası, clipboard/export riskleri ve client security standartları.                 | incelemelerinde frontend yüzeyi kontrol edilebilir olur.                          |
| 38 | PII / Data Classification | Alanların public/internal/confidential/PII gibi sınıflara ayrılması ve UI davranışlarının buna bağlanması. | Privacy Blur, export kısıtları ve hassas alan politikaları sistematik hale gelir. |
| 39 | Frontend Audit Hooks      | Kritik action ve mutationlarda backend audit için kullanıcı, ekran, kaynak ve değişiklik metadata üretimi. | Audit loglar yalnız endpoint değil gerçek kullanıcı bağlamını da taşır.           |
| 40 | Observability             | Error, API latency, render/performance, navigation ve kritik user-action telemetry.                        | Production problemleri ekran görüntüsü beklemeden teşhis edilebilir.              |
| 41 | Performance Budgets       | Bundle, route load, interaction latency, grid render ve long-task limitlerinin CI ile izlenmesi.           | Repo büyürken performansın sessizce bozulması engellenir.                         |
| 42 | Virtualization Strategy   | Büyük grid, list, dropdown ve tree yapılarında ortak virtualization yaklaşımı.                             | ERP ölçeğinde binlerce satır ve seçenek performanslı şekilde işlenir.             |
| 43 | Code Splitting Strategy   | Route, domain, extension ve ağır feature bazında lazy loading/code splitting standardı.                    | İlk açılış maliyeti kontrol altında kalır.                                        |

## 7. Test, Developer Experience ve Delivery Governance

Büyük ekip ve sık release temposunda mimariyi, kaliteyi ve dağıtım güvenliğini koruyan sistemler.

| #  | Katman             | Çalışma Mantığı                                                                            | Enterprise Katkısı                                                           |
|----|--------------------|--------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| 44 | Testing Pyramid    | Unit, integration, component, E2E, contract ve kritik akış testlerinin dengeli stratejisi. | Büyük refactor ve sık release süreçlerinde güven sağlar.                     |
| 45 | API Contract Tests | Frontend beklentileri ile backend/OpenAPI contractlarının CI içinde doğrulanması.          | Backend değişikliği productionda sessizce frontend kırmaz.                   |
| 46 | Visual Regression  | Kritik ekran ve design system componentlerinin otomatik screenshot karşılaştırmaları.      | Global stil veya component değişikliklerinin beklenmedik etkileri yakalanır. |

| #  | Katman                                  | Çalışma Mantığı                                                                                             | Enterprise Katkısı                                                            |
|----|-----------------------------------------|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| 47 | <b>Component Playground</b>             | Storybook veya benzeri izole component workbench ve durum senaryoları.                                      | Frontend geliştirme/QA için tüm ERPyi ayağa kaldırma ihtiyacı azalır.         |
| 48 | <b>Mock Server</b>                      | Gerçek backend bağımsız senaryo bazlı mock API ve fixture altyapısı.                                        | Frontend ve backend ekipleri paralel ilerler; edge-case testleri kolaylaşır.  |
| 49 | <b>Developer Diagnostics</b>            | Query inspector, permission inspector, feature flag inspector, event/debug panel gibi development araçları. | Mimari karmaşıklıktıkça debug maliyeti kontrol altında tutulur.               |
| 50 | <b>Dependency Rules</b>                 | Domainler arası import kurallarının lint/build araçlarıyla enforce edilmesi.                                | Mimari zaman içinde istemeden erozyona uğramaz.                               |
| 51 | <b>Public / Internal API Boundaries</b> | Her modülün dışarıya yalnız public API export etmesi; internal implementasyonun kapalı tutulması.           | Modüller arası gizli bağımlılıklar ve kırılğan refactorlar azalır.            |
| 52 | <b>Code Ownership</b>                   | Domain, platform package ve kritik componentler için sahiplik ve review kurallarının belirlenmesi.          | Büyük ekiplerde bakım ve mimari sorumluluk netleşir.                          |
| 53 | <b>CI Quality Gates</b>                 | Typecheck, lint, test, bundle budget, circular dependency, dead-code ve architecture checks.                | Kalite kuralları kişisel disiplin olmaktan çıkar, otomatik güvenceye dönüşür. |
| 54 | <b>Preview Environments</b>             | Her PR/branch için geçici çalışan frontend ve ilgili test ortamının sağlanması.                             | QA, ürün ve tasarım incelemeleri release öncesinde hızlanır.                  |
| 55 | <b>Controlled Rollout</b>               | Feature flag, canary, staged rollout, telemetry ve hızlı rollback kombinasyonu.                             | Enterprise production riski küçültülür.                                       |

## 8. White-Label, Offline, Veri Taşıma ve AI Hazırlığı

Ürünü partner/tenant varyasyonlarına, zayıf bağlantıya, yüksek hacimli veri taşımaya ve gelecekte agent tabanlı kullanıma hazırlayan katmanlar.

| #  | Katman                          | Çalışma Mantığı                                                                               | Enterprise Katkısı                                               |
|----|---------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| 56 | <b>White-Label Architecture</b> | Logo değişikliğinin ötesinde brand token, tenant theme, domain/config ve varyasyon altyapısı. | OEM/partner ve kurumsal marka senaryoları kontrollü desteklenir. |
| 57 | <b>Offline / Degraded Mode</b>  | Bağlantı sorunlarında cache, read-only erişim, draft koruma                                   | Saha ve zayıf bağlantı senaryolarında veri kaybı/iş              |

| #  | Katman                           | Çalışma Mantığı                                                                                          | Enterprise Katkısı                                                                              |
|----|----------------------------------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
|    |                                  | ve yeniden bağlanma stratejileri.                                                                        | kesintisi azalır.                                                                               |
| 58 | <b>Import / Export Framework</b> | CSV/XLSX mapping, validation, preview, error reporting ve background export için ortak framework.        | Her modül ayrı Excel/import kodu yazmak zorunda kalmaz.                                         |
| 59 | <b>Entity Reference System</b>   | Customer, Product, Invoice gibi entitylerin ortak id/type/display/reference kontratıyla temsil edilmesi. | Search, drawer, tabs, links, command palette ve AI aynı entity dilini paylaşır.                 |
| 60 | <b>AI-Ready UI Metadata</b>      | Action, entity, field, permission ve form metadata bilgilerinin machine-readable hale getirilmesi.       | Gelecekte AI agentlar DOMa bağımlı olmadan güvenli action/permission katmanlarını kullanabilir. |

## Önerilen Uygulama Sırası

60 capability aynı anda geliştirilmemeli. Önce kaldırıcı etkisi yüksek foundation katmanları kurularak sonraki featureların maliyeti düşürülmelidir.

| Faz                                          | Hedef                                                                                                                                                           |
|----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Faz 0 - Architecture Audit</b>            | Dependency graph, state management, routing, API erişimi, component coupling, bundle, test coverage ve mevcut extension noktalarının envanteri.                 |
| <b>Faz 1 - Platform Core</b>                 | App Shell, typed API, query/mutation standardı, permission, tenant context, feature flag, error architecture, telemetry ve security baseline.                   |
| <b>Faz 2 - Registry Architecture</b>         | Command, Action, Navigation, Route, Extension ve gerektiğinde Field/Capability registrylerinin kurulması.                                                       |
| <b>Faz 3 - Enterprise Data UX Foundation</b> | Grid abstraction, Filter AST, Saved View/View State, Selection Engine, Form Schema, Validation, Draft ve Overlay/Workspace sistemleri.                          |
| <b>Faz 4 - Scale &amp; Extensibility</b>     | Realtime events, background jobs, audit hooks, white-label/configuration, preview environments, rollout governance, offline/degraded mode ve AI-ready metadata. |

## Enterprise Seviyede Değişmez Kurallar

- **Backend authoritative kalır:** Optimistic UI dahil hiçbir client state finansal/iş kuralı açısından source of truth değildir.
- **Architecture rules CI ile enforce edilir:** Modül sınırları ve quality gate kuralları dokümanda kalmaz; lint/build/test ile zorlanır.
- **Business action bir kez tanımlanır:** Aynı aksiyon toolbar, grid, command palette veya AI için tekrar tekrar yazılmaz.
- **Server state merkezi yönetilir:** Liste, drawer, detay, dashboard ve realtime eventlerin aynı veri kaynağıyla reconcile edilmesi gerekir.
- **Permission yalnız gizleme değildir:** Frontend UX için permission kontrol eder; gerçek authorization her zaman backend tarafından enforce edilir.
- **Observability başlangıçtan vardır:** Production frontend hatası kullanıcı ekran görüntüsü beklenerek debug edilmemelidir.
- **Performance budget vardır:** Bundle ve interaction performansı feature sayısı arttıkça otomatik olarak korunmalıdır.
- **Extensibility kontrollüdür:** Extension/slot sistemi güçlüdür fakat açık contract, versioning ve permission sandbox olmadan müşteri kodu core'a sızmamalıdır.

**Hedef çıktı:** Sadece React ile yazılmış bir ERP arayüzü değil; yeni modüllerin, müşteri extensionlarının, yeni actionların ve gelecekte AI destekli iş akışlarının güvenli şekilde eklendiği kendi Enterprise Frontend Platformunuz.